



Computational and Applied Mathematics
Practical Project Report

Heat Transfer in Solids through Robin Boundary Conditions

Muhammad Nabeel, Aman Sayyad, Alin Siby

Supervisor:
Prof. Dr. Eberhard Bänsch

Summer Semester 2025

Contents

1	Introduction	3
2	Derivation of Robin Boundary Condition	3
2.1	Heat equation	3
2.2	Matching Conditions	4
2.3	Modified Boundary Layer	5
2.4	Effective Boundary Condition	7
3	Numerical Process	9
3.1	Finite Element Method (FEM)	9
4	Programming	9
4.1	Main Code	9
4.2	Code Validation	12
5	Parameter Analysis	16
5.1	Effect of Heat Transfer Coefficient h	16
6	Conclusion	19

1 Introduction

The cooling of metals using fluids is a critical industrial process, widely applied in areas such as quenching, industrial cooling systems, and heat exchangers. In such scenarios, the heat exchange at the solid–fluid interface is modeled using the Robin boundary condition, which states that the heat flux at the boundary is a linear function of boundary temperature. [2]

In this project, we consider a two-dimensional rectangular solid domain of length l and height K and analyze steady-state heat conduction within it. Starting from the transmission conditions (continuity of temperature and heat flux across an interface) we rigorously derive the Robin boundary condition. This condition captures convective heat exchange between the solid and its surrounding environment.

Then, we solve the steady-state heat equation using MATLAB’s PDE Toolbox. After verifying the correctness of the implementation, we perform a parameter study to investigate the effect on temperature at the interface by varying the heat transfer coefficient.

2 Derivation of Robin Boundary Condition

2.1 Heat equation

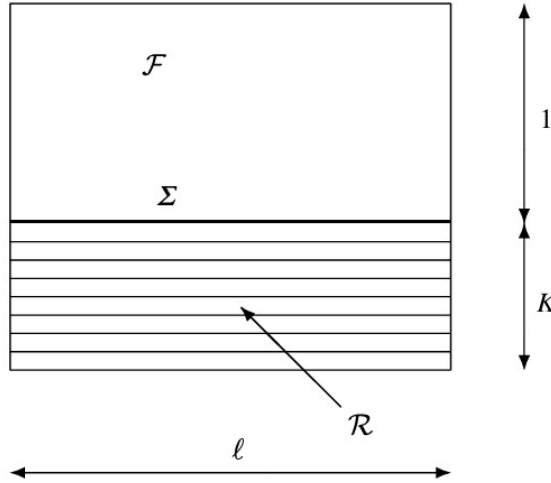


Figure 1: Rectangular Domain Ω with rigid $\mathcal{R}$ and fluid $\mathcal{F}$ parts [2]

Let $\Omega \subset \mathbb{R}^2$, be a bounded simply connected domain separated into two parts $\Omega = R \cup F$, both being simply connected domains, separated by a smooth bounded interface $\Sigma = \bar{R} \cap \bar{F}$.

Here, R represents a rigid body and F the surrounding medium, filled by some fluid, cooling the body R . where:

$$\begin{aligned} R &= (0, l) \times (-K, 0) \\ F &= (0, l) \times (0, 1) \\ \Sigma &= \Omega \cap \{y = 0\} \end{aligned}$$

We model the heat equations as follows.

$$-\alpha \Delta u = 0 \quad \text{in } R, \quad (1)$$

$$-\beta \Delta u + \mathbf{v} \cdot \nabla u = 0 \quad \text{in } F, \quad (2)$$

$$u(x, 1) = T, \quad (3)$$

$$u(x, -K) = W \text{ and } u(0, y) = u(1, y) = W \quad \text{for } y \in (-K, 0), \quad (4)$$

$$u(0, y) = u(l, y) = 0 \quad \text{for } 1 > y > 0, \quad (5)$$

$$u(x, 0-) = u(x, 0+), \quad (6)$$

$$\alpha \frac{\partial u}{\partial y}(x, 0-) = \beta \frac{\partial u}{\partial y}(x, 0+) - v_y u(x, 0). \quad (7)$$

For simplicity, we assume that the convection velocity is constant:

$$\mathbf{v} = (v_x, v_y) = \text{const.}$$

Now if we consider the equations (2), (3), (5), (6) we get a non-homogeneous boundary value problem for u in F . If we assume that we know $u(x, 0-)$ then we can think our solution temperature u to be of the form

$$u = b + T$$

for a constant T . Where b is of the form

$$-\beta \Delta b + v \cdot \nabla b = 0 \quad \text{in } F \quad (8)$$

$$b(x, 1) = 0 \text{ \& } b(0, y) = b(l, y) = -T \quad \text{for } 1 > y > -k \quad (9)$$

$$b(x, 0) = u(x, 0-) - T \quad (10)$$

If for such b , we can prove that $\frac{\partial b}{\partial y}(x, 0-) = H(u(x, 0+) - T)$ and for some constant H , then we have our Robin Boundary condition on the interface Σ .

For simplicity, we assume

$$T = 0 \implies u(x, 1) = 0.$$

2.2 Matching Conditions

Now, we introduce two constants to make our calculations easier.

$$q = \frac{v_x}{2\beta}, \quad (11)$$

$$p = \frac{v_y}{2\beta}. \quad (12)$$

and the continuity equations (6) and (7) would read

$$u(x, 0+) = u(x, 0-) \implies b(x, 0) = u(x, 0-), \quad (13)$$

$$\beta \frac{\partial u}{\partial y}(x, 0+) - v_y u(x, 0) = \alpha \frac{\partial u}{\partial y}(x, 0-) \implies \beta \left[\frac{\partial b}{\partial y}(x, 0+) - 2p b(x, 0) \right] = \alpha \frac{\partial u}{\partial y}(x, 0-). \quad (14)$$

Therefore, equations particular to the fluid region look like this.

$$-\beta \Delta b + \mathbf{v} \cdot \nabla b = 0 \quad \text{in } F, \quad (15)$$

$$b(x, 1) = 0, \quad (16)$$

$$b(0, y) = b(l, y) = 0 \quad \text{for } 1 > y > -k, \quad (17)$$

$$b(x, 0) = u(x, 0-), \quad (18)$$

$$\beta \left[\frac{\partial b}{\partial y}(x, 0+) - 2p b(x, 0) \right] = \alpha \frac{\partial u}{\partial y}(x, 0-). \quad (19)$$

Now, for the Σ we have two equations [(18) and (19)] and it might be difficult to find a solution for such a system. Therefore we choose to assume $u(x, 0-)$ is known. Then $b(x, 0)$ and $\frac{\partial b}{\partial y}$ depend on $u(x, 0-)$. And (15), (16), (17) have the following properties.

1. The PDE in (15) is linear and homogeneous.
2. The boundary conditions (16) and (17) are linear and homogeneous.

Thus (15), (16), (17), can be solved using the Fourier's Method.

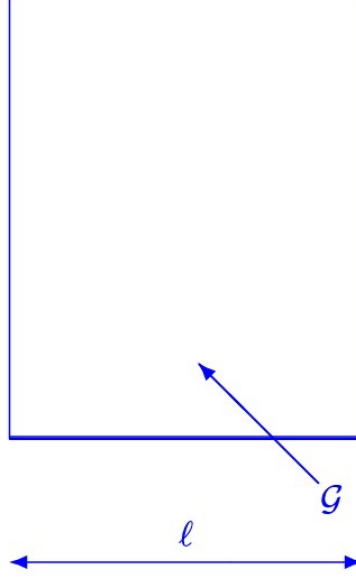


Figure 2: Infinity Strip for Boundary Layer Corrector
[2]

2.3 Modified Boundary Layer

The problem is now modified to focus on the boundary layer near $y=0$, making the analysis simpler and more local. This helps study how the fluid solution depends on the interface condition.

$$-\Delta b + 2q \frac{\partial b}{\partial x} + 2p \frac{\partial b}{\partial y} = 0 \quad \text{in } G = (0, l) \times (0, +\infty), \quad (20)$$

$$b(0, y) = b(l, y) = 0 \quad (21)$$

$$\lim_{y \rightarrow +\infty} b = 0 \quad (22)$$

$$b(x, 0) = u(x, 0-). \quad (23)$$

Since the heat equation and the boundary conditions are linear and homogeneous, we can use Fourier's Method to solve the pde. We look for b in the separated form:

$$b(x, y) = X(x)Y(y). \quad (24)$$

Substituting this b in (20) gives

$$\begin{aligned} -\Delta(XY) + 2q \frac{\partial}{\partial x}(XY) + 2p \frac{\partial}{\partial y}(XY) &= 0 \\ \implies -X''Y - XY'' + 2qX'Y + 2pXY' &= 0 \\ \implies -X(Y'' - 2pY') - Y(X'' - 2qX') &= 0 \end{aligned}$$

$$\frac{Y'' - 2pY'}{Y} = \frac{-X'' + 2qX'}{X} \quad (25)$$

The LHS of (25) is a function in y and the RHS is a function in x thus they must be equal to some constant λ^2 . Thus (25) reads as

$$\frac{Y'' - 2pY'}{Y} = \frac{-X'' + 2qX'}{X} = \lambda^2 \quad \text{or} \quad (26)$$

$$Y'' - 2pY' - \lambda^2 Y = 0 \quad \text{and} \quad X'' - 2qX' + \lambda^2 X = 0 \quad (27)$$

Since Y is a linear ordinary differential equation of 2nd order and $b^2 - 4ac > 0$ for $a=1$, $b=-2p$ and $c = -\lambda^2$ its general solution is of the form

$$Y = c_1 e^{s_1 y} + c_2 e^{s_2 y} \quad (28)$$

with s_1 and s_2 are real and distinct roots of the characteristic equation

$$s^2 - 2ps - \lambda^2 = 0 \quad (29)$$

The roots of (29) are as follows

$$s_{1,2} = p \pm \sqrt{p^2 + \lambda^2}. \quad (30)$$

Although s_1 is a mathematical solution, it is not physically admissible and is therefore excluded. In correspondence with equation (22) we take the negative root s_2 from (30), that is, we want

$$\lim_{y \rightarrow \infty} b = 0$$

since

$$\begin{aligned} \lim_{y \rightarrow \infty} b &= \lim_{y \rightarrow \infty} X(x)Y(y) \\ \implies \lim_{y \rightarrow \infty} b &= X(x) \lim_{y \rightarrow \infty} e^{s_2 y} \end{aligned}$$

only possible when $s = s_2 = p - \sqrt{p^2 + \lambda^2}$.

For $b^2 - 4ac \geq 0$ only trivial solution follow the required boundary condition $X(0) = 0$ and $X(l) = 0$. Therefore, for $b^2 - 4ac < 0$ where $a=1$, $b = -2q$ and $c = \lambda^2$, the general solution for X has the form:

$$X(x) = e^{\gamma x} (A \sin \sigma x + B \cos \sigma x), \quad (31)$$

The characteristic equation for X is

$$r^2 - 2qr + \lambda^2 = 0. \quad (32)$$

And its complex roots of the form $r = \gamma \pm i\sigma$ are given by

$$r_{1,2} = q \pm \sqrt{q^2 - \lambda^2},$$

$$r_{1,2} = q \pm i\sqrt{\lambda^2 - q^2}$$

where $\gamma = q$ and we know that

$$\begin{aligned} X(0) &= e^{\gamma 0} (A \sin(\sigma 0) + B \cos(\sigma 0)) = 0 \\ &\implies B = 0 \\ \implies X(x) &= e^{\gamma x} (A \sin(\sigma x)) \\ X(l) &= e^{\gamma l} A \sin(\sigma l) = 0 \end{aligned}$$

for all $A \neq 0$ we have $\sigma = \frac{k\pi}{l}$, $k \in \mathbb{N}$. Thus,

$$\sqrt{\lambda^2 - q^2} = \frac{k\pi}{l},$$

$$\lambda^2 - q^2 = \frac{k^2\pi^2}{l^2},$$

and

$$\lambda^2 = \lambda_k^2 = q^2 + \frac{k^2\pi^2}{l^2}. \quad (33)$$

Consequently,

$$s_k = p - \sqrt{p^2 + \lambda_k^2} = p - \sqrt{p^2 + q^2 + \frac{k^2\pi^2}{l^2}},$$

so that, denoting $w^2 = p^2 + q^2$,

$$\begin{aligned} \implies s_k &= p - \sqrt{w^2 + \left(\frac{k\pi}{l}\right)^2} \\ \implies s_k &= p - \sqrt{w^2 + w^2\left(\frac{k\pi}{wl}\right)^2} \\ s_k &= p - w\sqrt{1 + \left(\frac{k\pi}{wl}\right)^2}. \end{aligned} \quad (34)$$

Therefore, the solution

$$b = X(x)Y(y)$$

now has the form:

$$b = e^{sy} e^{qx} A \sin \sigma x$$

We neglect the cosine term because $X(0)=X(l)=0$. After we sum over k and get the final form as

$$b(x, y) = e^{qx} \sum_{k=1}^{\infty} e^{s_k y} A_k \sin\left(\frac{k\pi x}{l}\right), \quad (35)$$

where the Fourier coefficients A_k are computed from eq(23) as

$$\sum_{k=1}^{\infty} A_k \sin\left(\frac{k\pi x}{l}\right) = u(x, 0-) e^{-qx}. \quad (36)$$

$$A_k = \frac{2}{l} \int_0^l u(t, 0-) e^{-qt} \sin\left(\frac{k\pi t}{l}\right) dt, \quad k = 1, 2, 3, \dots \quad (37)$$

2.4 Effective Boundary Condition

From (35) we know that

$$b(x, y) = e^{qx} \sum_{k=1}^{\infty} e^{s_k y} A_k \sin\left(\frac{k\pi x}{l}\right)$$

therefore,

$$\frac{\partial b(x, y)}{\partial y} = e^{qx} \sum_{k=1}^{\infty} s_k e^{s_k y} A_k \sin\left(\frac{k\pi x}{l}\right) \quad (38)$$

$$\frac{\partial b(x, 0+)}{\partial y} = e^{qx} \sum_{k=1}^{\infty} s_k A_k \sin\left(\frac{k\pi x}{l}\right) \quad (39)$$

From (34) we know that

$$s_k = p - w\sqrt{1 + \left(\frac{k\pi}{wl}\right)^2}$$

using the series expansion of $\sqrt{1 + \left(\frac{k\pi}{l}\right)^2}$ we get

$$s_k = p - w\left(1 + \frac{1}{2}\left(\frac{k\pi}{wl}\right)^2 - \frac{1}{8}\left(\frac{k\pi}{wl}\right)^4 + \dots\right)$$

$$s_k = p - w\left(1 + \frac{1}{2}\left(\frac{k\pi}{wl}\right)^2 + O\left(\frac{k\pi}{wl}\right)^4\right) \quad (40)$$

Substituting (40) in (39) we get

$$\begin{aligned} \frac{\partial b(x, 0+)}{\partial y} &= (p - w)e^{qx} \sum_{k=1}^{\infty} A_k \sin \frac{k\pi x}{l} - \frac{1}{2w}e^{qx} \sum_{k=1}^{\infty} \left(\frac{k\pi}{l}\right)^2 A_k \sin \frac{k\pi x}{l} \\ &\quad + \frac{1}{w^3}e^{qx} \sum_{k=1}^{\infty} O\left(\left(\frac{k\pi}{l}\right)^4\right) A_k \sin \frac{k\pi x}{l} \\ &= (p - w)u(x, 0-) - \frac{1}{2w}e^{qx} \sum_{k=1}^{\infty} \left(\frac{k\pi}{l}\right)^2 A_k \sin \frac{k\pi x}{l} + \frac{1}{w^3}e^{qx} \sum_{k=1}^{\infty} O\left(\left(\frac{k\pi}{l}\right)^4\right) A_k \sin \frac{k\pi x}{l} \end{aligned} \quad (41)$$

From (36), we know that

$$\sum_{k=1}^{\infty} A_k \sin \left(\frac{k\pi x}{l}\right) = u(x, 0-)e^{-qx}$$

Deriving twice with respect to x gives (we assume $u \in C^\infty(\Sigma)$)

$$\begin{aligned} \frac{\partial^2}{\partial x^2} \left[\sum_{k=1}^{\infty} A_k \sin \left(\frac{k\pi x}{l}\right) \right] &= \frac{\partial^2}{\partial x^2} u(x, 0)e^{-qx} \\ \Rightarrow \sum_{k=1}^{\infty} \left(\frac{k\pi}{l}\right)^2 A_k \sin \left(\frac{k\pi x}{l}\right) &= -e^{-qx} \left(\frac{\partial^2}{\partial x^2} u(x, 0) - 2q \frac{\partial}{\partial x} u(x, 0) + q^2 u(x, 0) \right) \end{aligned} \quad (42)$$

Substituting (42) in (41) we get the following result

$$\begin{aligned} \frac{\partial b(x, 0+)}{\partial y} &= (p - w)u(x, 0) - \frac{1}{2w}e^{qx} \left[-e^{-qx} \left(\frac{\partial^2}{\partial x^2} u(x, 0) - 2q \frac{\partial}{\partial x} u(x, 0) + q^2 u(x, 0) \right) \right] \\ &\quad + \frac{1}{w^3}e^{qx} \sum_{k=1}^{\infty} O\left(\left(\frac{k\pi}{l}\right)^4\right) A_k \sin \frac{k\pi x}{l} \\ &= \left(p - w + \frac{q^2}{2w} \right) u(x, 0) - \frac{q}{w} \frac{\partial}{\partial x} u(x, 0) + \frac{1}{2w} \frac{\partial^2}{\partial x^2} u(x, 0) + O\left(\frac{q^4}{w^3} + \frac{1}{w^3}\right). \end{aligned} \quad (43)$$

Substituting (43) in (19) we get

$$\frac{\beta}{\alpha} \left[\left(-p - w + \frac{q^2}{2w} \right) u(x, 0) - \frac{q}{w} \frac{\partial u}{\partial x}(x, 0) + \frac{1}{2w} \frac{\partial^2 u}{\partial x^2}(x, 0) + O\left(\frac{q^4}{w^3} + \frac{1}{w^3}\right) \right] = \frac{\partial u}{\partial y}(x, 0) \quad (44)$$

We make assumptions on α , β , p , and q which lead us to the Robin Boundary Condition. To justify the above computation, we need a non-trivial convection velocity. Therefore

$$w \neq 0 \iff (q, p) \neq (0, 0) \quad (45)$$

And for $p > 0$ and $q \ll p$ and (since $w^2 = p^2 + q^2$), $\beta \ll \alpha$, the effective boundary conditions are of the Robin type.

$$\frac{\partial u}{\partial y}(x, 0) = -Hu(x, 0) \quad \text{with} \quad (46)$$

$$H = \frac{\beta}{\alpha}(w + p) > 0 \quad (47)$$

Here, α denotes the thermal conductivity of the solid, while the term $\beta(w + p)$ is commonly referred to by physicists as the *heat transfer coefficient*. This coefficient is typically determined through experimental measurements. [2]

3 Numerical Process

To solve the above problem, we use the built-in MATLAB PDE solver, which is available through the `solvepde` function in the PDE Toolbox. This toolbox employs the Finite Element Method (FEM) to numerically solve partial differential equations.

3.1 Finite Element Method (FEM)

The FEM involves the following main steps:

1. Discretize the domain into smaller finite elements (mesh generation).
2. Derive the finite element equations for each element based on the weak formulation of the PDE.
3. Assemble the global system of equations by combining the element equations.
4. Apply the boundary conditions to modify the system accordingly.
5. Solve the resulting system of algebraic equations for the unknowns.
6. Visualize the numerical solution using contour plots, surface plots, or other graphical representations.

4 Programming

Thus, we incorporate the following system of steady-state heat transfer equations in the solid domain in MATLAB,

$$-\alpha \Delta u = 0 \quad \text{in } (0, 1) \times (-1, 0) \quad (48)$$

$$u(x, y) = W, \quad \text{on } x = 0, x = 1 \text{ and } y \in (-1, 0) \quad (49)$$

$$u(x, y) = W \quad \text{on } y = -1 \text{ and } x \in [0, 1] \quad (50)$$

$$\alpha \frac{\partial u}{\partial y} + hu = hT \quad \text{on } y = 0 \text{ and } x \in (0, 1) \quad (51)$$

and compute the temperature 'u' using the MATLAB PDE TOOLBOX [3].

4.1 Main Code

We run the code for the following physical experiment.

- 2-D Copper Block
- Thermal conductivity $\alpha = 401 \frac{W}{mK} [1]$
- Heat Transfer Coefficient $h = 25 \frac{W}{m^2K} [1]$

- Temperature on Dirichlet Boundary $W=500K$
- Temperature of Surrounding Fluid $T=300K$

```
% Create PDE model
model = createpde();

% Define square domain [0,1] x [-1,0]
R = [3 4 0 1 1 0 -1 -1 0 0]'; % 3: rectangle, 4: corners, x: 0→1, y: -1→0
gd = R;
sf = 'R';
ns = char('R');
ns = ns';
geometryFromEdges(model, decsg(gd, sf, ns));
pdegplot(model, EdgeLabels="on") % for 2-D

% PDE coefficients for Laplace equation
%  $m(\frac{\partial^2 u}{\partial t^2}) + d(\frac{\partial u}{\partial t}) - c(\Delta u) + au = f$ 
specifyCoefficients(model, m= 0, d=0, c=401, a=0, f=0);

% Apply Dirichlet boundary conditions on 3 edges e.g on 1,2,and 4.
% for dirichlet boundary MATLAB has the form hu=r
% specify u or (h,r) when needed
applyBoundaryCondition(model, 'dirichlet', Edge=1:2, r=500, h=1);
applyBoundaryCondition(model, 'dirichlet', Edge=4, r=500, h=1);
% General Neumann Boundary in MATLAB  $n \cdot (c \nabla u) + qu = g$ ;
% Apply Robin boundary condition  $c \cdot du/dy + qu = g$  on edge 3
applyBoundaryCondition(model, 'neumann', Edge=3, g=25*300, q=25);
% Generate mesh
generateMesh(model, Hmax=0.01, GeometricOrder='quadratic');

% Solve the PDE
result = solvepde(model);
u_num = result.NodalSolution;

% Plot numerical solution
figure(1);
pdeplot(model, XYData=u_num, ZData=u_num, Mesh="off");
title('Numerical Solution: u(x,y)');
xlabel('x'); ylabel('y'); zlabel('u');
```

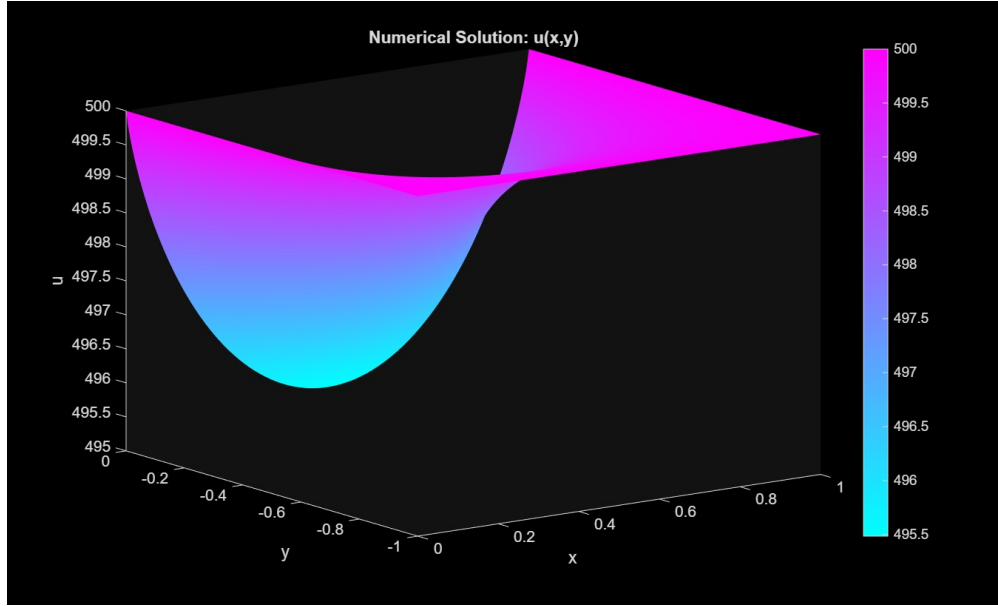


Figure 3: The temperature behavior of Copper for $\alpha = 401 \frac{W}{mK}$ and $h = 25 \frac{W}{m^2K}$ (Mesh off)

Output

- Maximum Temperature : 500K
- Minimum Temperature : 495.5K

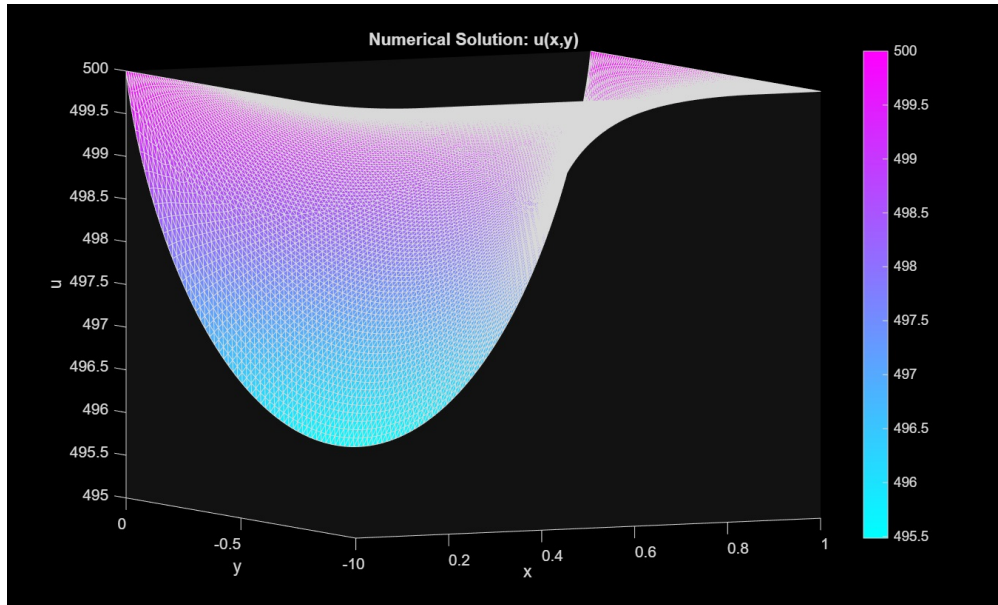


Figure 4: The temperature behavior of Copper for $\alpha = 401 \frac{W}{mK}$ and $h = 25 \frac{W}{m^2K}$ (Mesh on)

4.2 Code Validation

To verify the correctness of our MATLAB code, we used the method of manufactured solutions. We selected the artificial (analytical) function:

$$u(x, y) = x^3 + y^3$$

as the exact solution to the PDE. Substituting this into the Laplace equation with constant c , we derived the corresponding source term:

$$-\nabla \cdot (\alpha \nabla u) = f(x, y).$$

Since

$$\nabla u = \begin{bmatrix} 3x^2 \\ 3y^2 \end{bmatrix}, \quad \Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 6x + 6y,$$

the source term becomes:

$$f(x, y) = -\alpha \Delta u = -\alpha(6x + 6y).$$

On the boundary where a Robin condition is applied ($y=0$), we used the general form:

$$\alpha \frac{\partial u}{\partial y} + hu = g(x, y),$$

where $\frac{\partial u}{\partial y}$ is the normal derivative of u on the boundary. For each boundary segment, this term was computed using the known expression for u , and the function $g(x, y)$ was constructed accordingly:

$$g(x, y) = \alpha \frac{\partial u}{\partial y} + h(x^3 + y^3).$$

By using the parameters $\alpha = 401 \frac{W}{mK}$ of copper and $h = 25 \frac{W}{m^2K}$. We measured the maximum absolute error between the numerical and exact solution for various mesh sizes $H_{\max}$.

```
% Create PDE model
model = createpde();

% Define square domain [0,1] x [-1,0]
R = [3 4 0 1 1 0 -1 -1 0 0]'; % 3: rectangle, 4: corners, x: 0→1, y: -1→0
gd = R;
sf = 'R';
ns = char('R');
ns = ns';
geometryFromEdges(model, decsg(gd, sf, ns));
pdegplot(model, EdgeLabels="on") % for 2-D

% PDE coefficients for Laplace equation
% m(\pdv2{u}{t}) + d(\pdv{u}{t}) - c(\Delta u) + au = f
funct2=@(location,state) -401.*(location.x.*6+location.y.*6);
specifyCoefficients(model, m= 0, d=0, c=401, a=0, f=funct2);

% Apply Dirichlet boundary conditions on edges 1 to 3
boundaryFunc = @(location, state) location.x.^3+location.y.^3;
% for dirichlet boundary MATLAB has the form hu=r
% specify u or (h,r) when needed
% applyBoundaryCondition(model,"dirichlet",Edge=1:4,r=3,h=1);
applyBoundaryCondition(model, 'dirichlet',Edge=1:2,r=boundaryFunc,h=1);
applyBoundaryCondition(model, 'dirichlet',Edge=4,r=boundaryFunc,h=1);
% Genaral Neumann Boundary in MATLAB n . ( c \nabla u ) + qu = g;
% Apply Robin boundary condition du/dy + qu = g on edge 3
```

```

funct=@(location,state) 401*3.*location.y.^2+25.*(location.x.^3+location.y.^3);
applyBoundaryCondition(model, 'neumann', Edge=3,g=funct,q=25);

hmax_values = [0.1, 0.05, 0.025, 0.0125,0.00625,0.003125]; % Different mesh sizes
max_errors = zeros(size(hmax_values));
E=[];

for i = 1:length(hmax_values)
    h1 = hmax_values(i);
    % Generate mesh
    generateMesh(model,Hmax=h1, GeometricOrder='quadratic');

    % Solve the PDE
    result = solvepde(model);
    u_num = result.NodalSolution;

    u_exact_f = @(x, y) y.^3 + x.^3;

    % Evaluate exact solution at mesh nodes: u(x,y) = function of x and y
    nodes = model.Mesh.Nodes; % 2 x N matrix
    x_nodes = nodes(1,:); % x-coordinates as column
    y_nodes = nodes(2,:); % y-coordinates as column
    u_exact = u_exact_f(x_nodes, y_nodes);

    % Compute absolute error
    error = abs(u_exact - u_num);
    % Store max errors
    max_errors(i) = max(error);
    E=max_errors;
    fprintf('Hmax = %.5f → Max error = %.5e\n', h1, max_errors(i));
end
diff_max_errors = abs(diff(max_errors));
rel_diff_percent = (diff_max_errors ./ max_errors(1:end-1)) * 100;

fprintf('\nDifferences between successive max errors:\n');
for i = 1:length(diff_max_errors)
    fprintf('From Hmax=%.4f to %.4f: |max_error|=%.5e\n', ...
        hmax_values(i), hmax_values(i+1), diff_max_errors(i));
end

order=log(E(1:end-1)./E(2:end))./log(hmax_values(1:end-1)./hmax_values(2:end))

% Plot numerical solution
figure(1);
pdeplot(model, XYData=u_num, ZData=u_num, Mesh="off");
title('Numerical Solution: u(x,y)');
xlabel('x'); ylabel('y'); zlabel('u');

% Plot exact solution
figure(2);
pdeplot(model, XYData=u_exact, ZData=u_exact, Mesh="off");
title('Exact Solution: u(x,y) = x^3+y^3');
xlabel('x'); ylabel('y'); zlabel('u');

```

```

% Plot error
figure(3);
pdeplot(model, XYData=error, ZData=error, Mesh="off");
title('Absolute Error |u_{exact} - u_{num}|');
xlabel('x'); ylabel('y'); zlabel('Error');

```

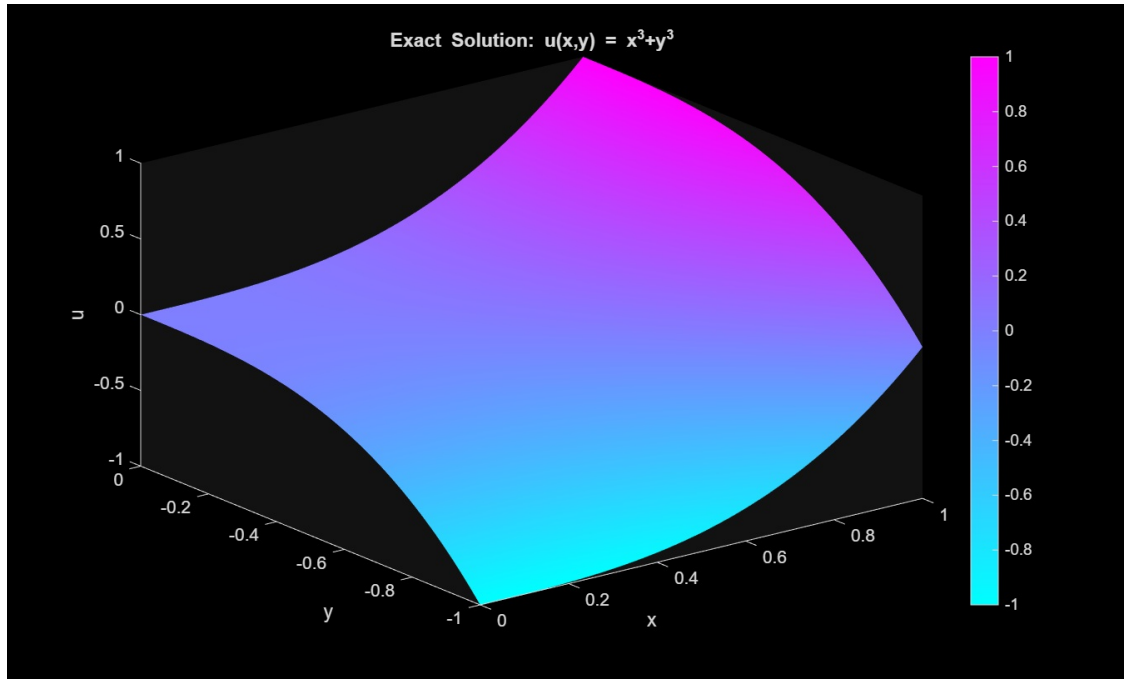


Figure 5: Exact Solution

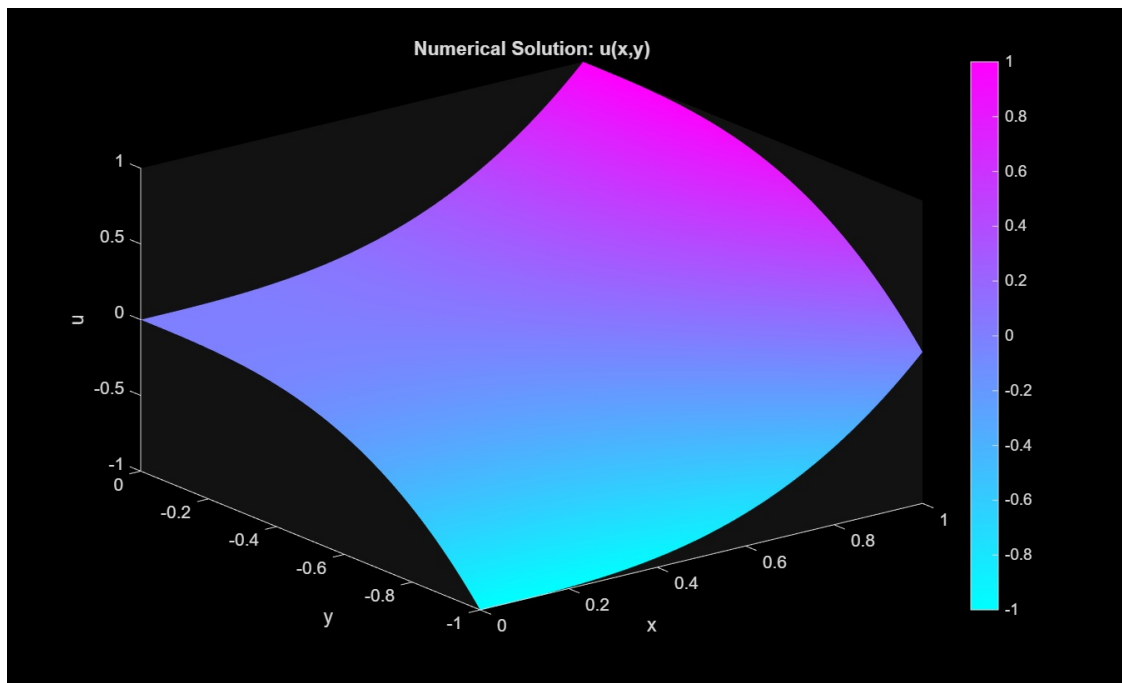


Figure 6: Numerical Solution

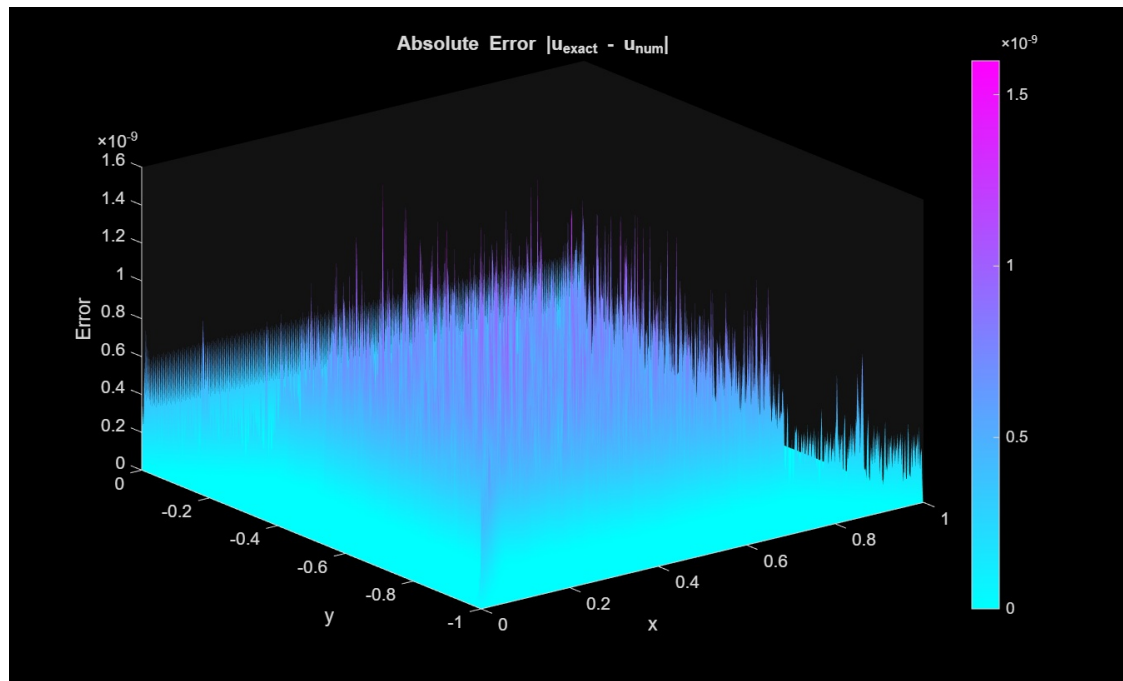


Figure 7: Error Plot

Table 1: Maximum error and estimated order of convergence for decreasing mesh size

$H_{\max}$	Max Error : $E_{H_{\max}(i)}$	Estimated Order : $E_0 = \frac{\log\left(\frac{E_{H_{\max}(i)}}{E_{H_{\max}(i+1)}}\right)}{\log\left(\frac{H_{\max}(i)}{H_{\max}(i+1)}\right)}$
0.10000	2.61360×10^{-5}	—
0.05000	3.06547×10^{-6}	3.0919
0.02500	4.07688×10^{-7}	2.9106
0.01250	5.15053×10^{-8}	2.9847
0.00625	1.04213×10^{-8}	2.3052
0.00313	1.59878×10^{-9}	2.7045

The observed convergence orders indicate that the numerical scheme exhibits approximately second to third order accuracy, consistent with the use of quadratic finite elements and a sufficiently smooth solution.

5 Parameter Analysis

5.1 Effect of Heat Transfer Coefficient h

We investigate how varying the heat transfer coefficient h influences the temperature distribution in various solid materials. By increasing h from $25 \frac{W}{m^2K}$ to $500 \frac{W}{m^2K}$, we capture the transition from weak to strong convective cooling at the boundary.

As expected, higher values of h lead to greater heat dissipation through the boundary, resulting in a lower minimum temperature within the solid. This behavior reflects the growing efficiency of the boundary in transferring heat as h increases.

To better understand this effect, we examine the relationship between the minimum internal temperature and the heat transfer coefficient across different materials, including copper, iron, silicon nitride, clay, and brick. This comparative study reveals how each material responds to convective cooling and offers insight into their relative thermal performance under the Robin Boundary condition.

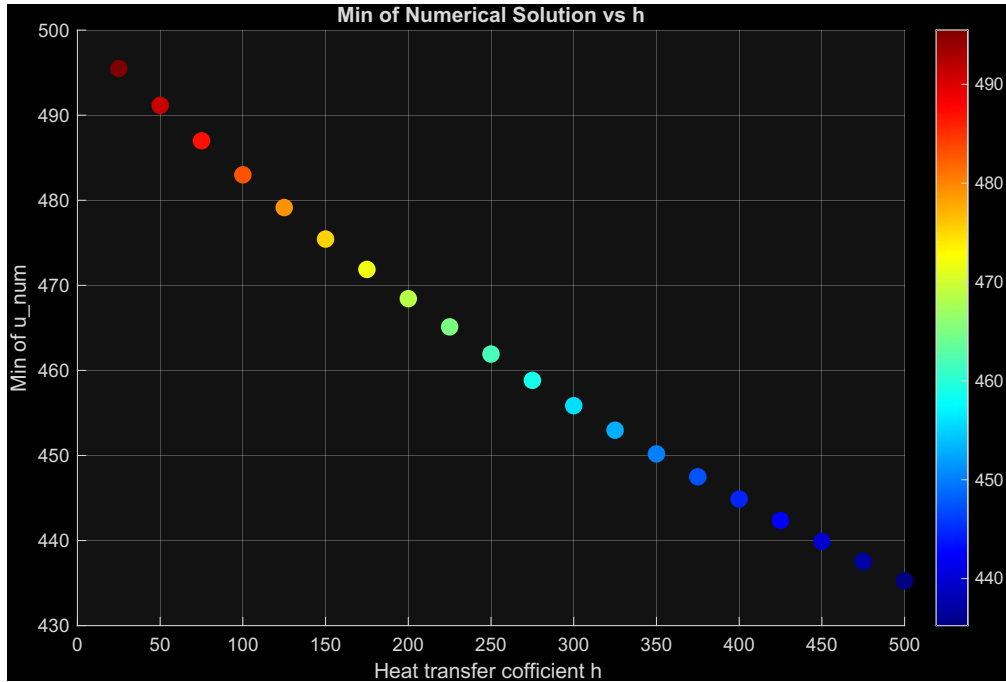


Figure 8: For copper $\alpha = 401 \frac{W}{mK}$

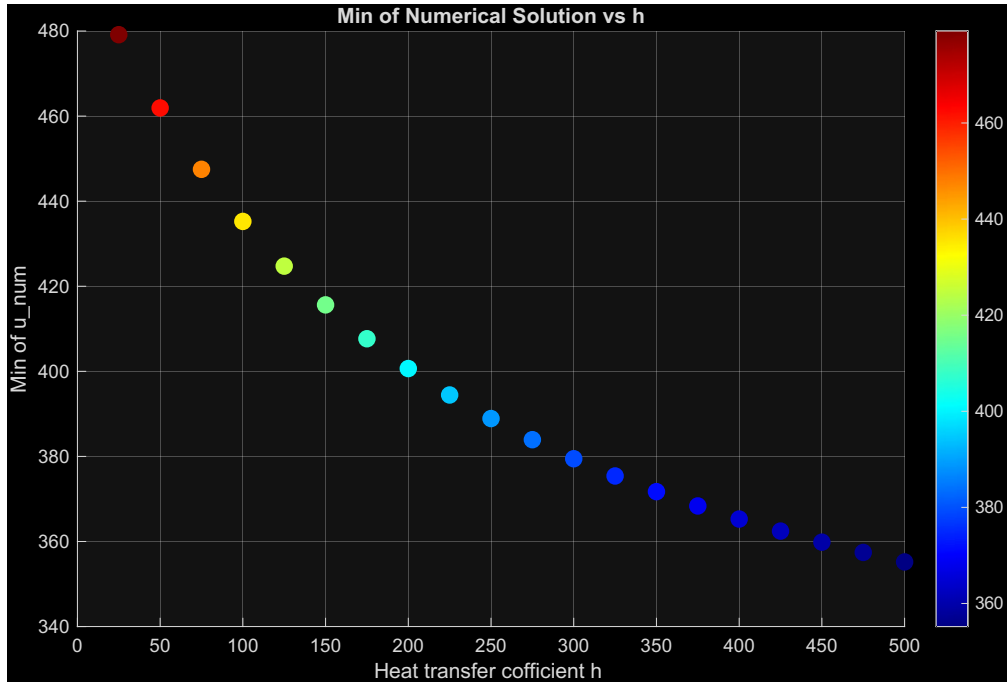


Figure 9: For Iron $\alpha = 80.2 \frac{W}{mK}$
[1]

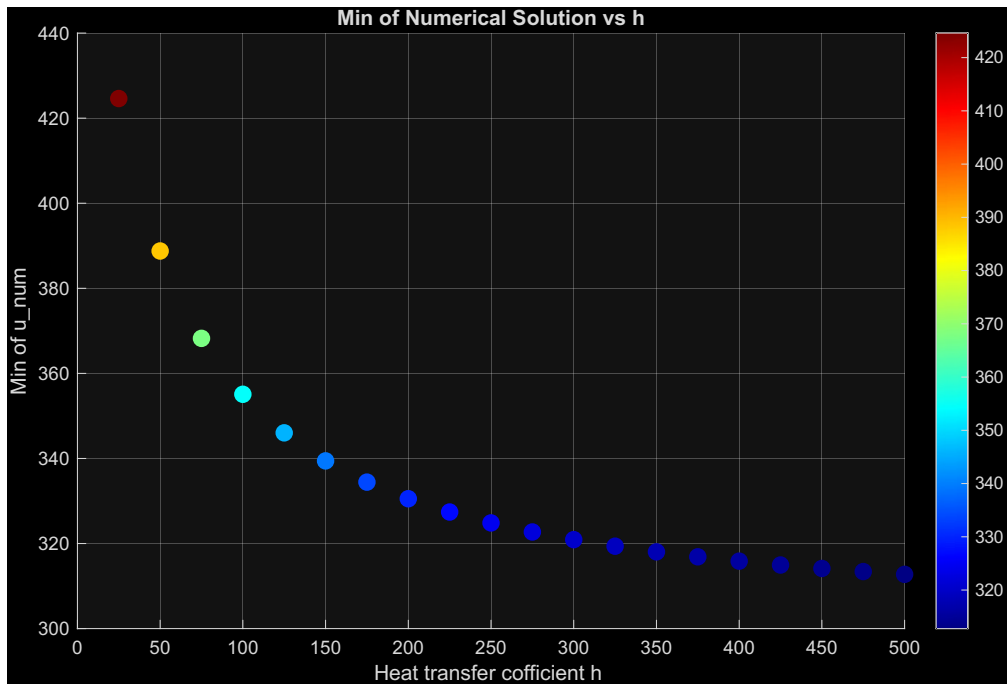


Figure 10: For Silicon nitride $\alpha = 16 \frac{W}{mK}$
[1]

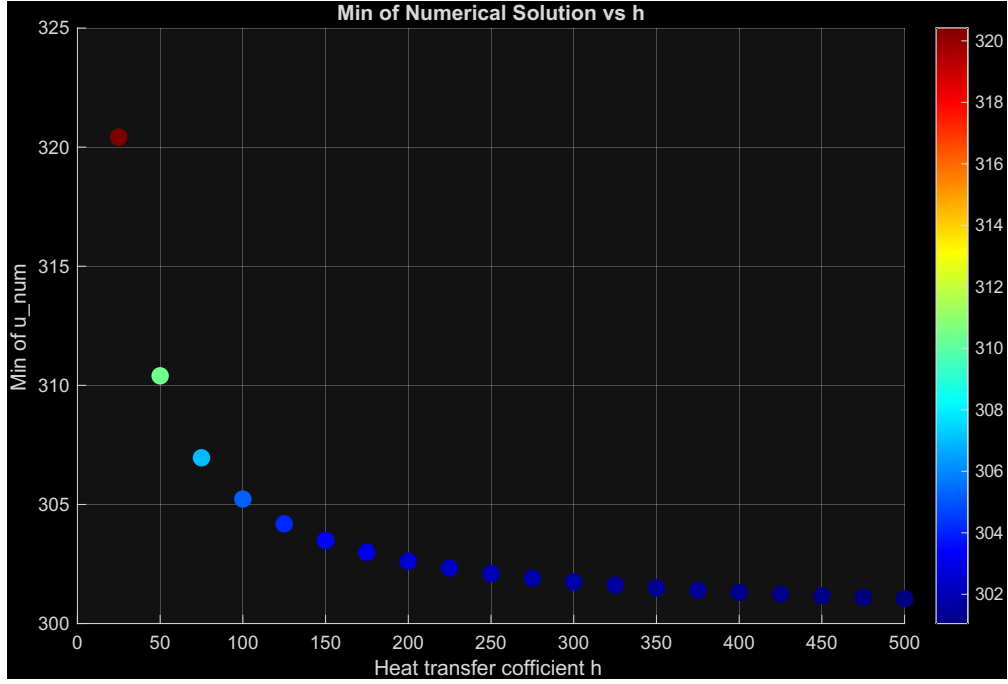


Figure 11: For Clay $\alpha = 1.3 \frac{W}{mK}$
[1]

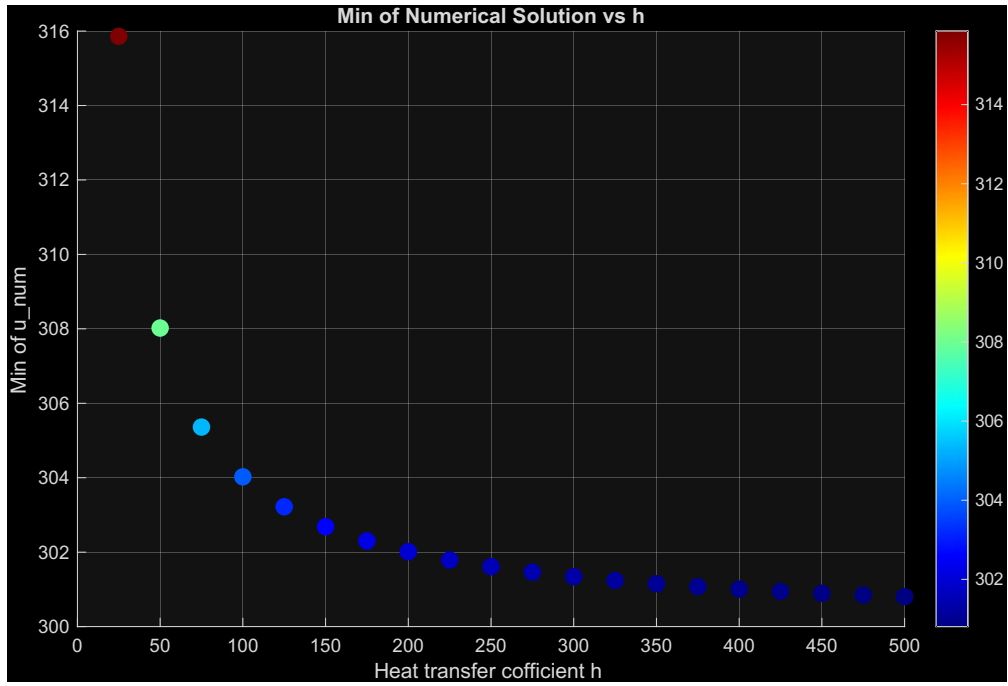


Figure 12: For Brick $\alpha = 1 \frac{W}{mK}$
[1]

6 Conclusion

In this project, we derived the Robin boundary condition from fundamental transmission conditions, namely the continuity of temperature and heat flux across an interface. We implemented the resulting steady-state heat conduction model in MATLAB using its PDE Toolbox, and validated the correctness of the numerical solution through consistency checks and limiting behavior.

To examine the physical relevance of the model, we performed simulations using material properties corresponding to iron, copper, silicon nitride, clay, and brick. These tests allowed us to explore how variations in thermal conductivity influence the behavior of the Robin boundary. Our results aligned with physical expectations, confirming that the boundary condition correctly transitions between Dirichlet and Neumann like behavior depending on the parameter values.

This study highlights the usefulness of Robin boundary conditions in modeling realistic thermal interactions at solid–fluid interfaces and demonstrates how numerical simulations can be used to validate and interpret boundary-driven phenomena in steady-state heat transfer.

References

- [1] Theodore L Bergman. *Fundamentals of heat and mass transfer*. John Wiley & Sons, 2011.
- [2] Eduard Marušić-Paloka and Igor Pažanin. “The Robin boundary condition for modelling heat transfer”. In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 480 (2024), p. 20230850. DOI: 10.1098/rspa.2023.0850.
- [3] The MathWorks, Inc. *Partial Differential Equation Toolbox – MATLAB & Simulink*. Online. Accessed July 29, 2025. 2025. URL: <https://ww2.mathworks.cn/en/products/pde.html>.